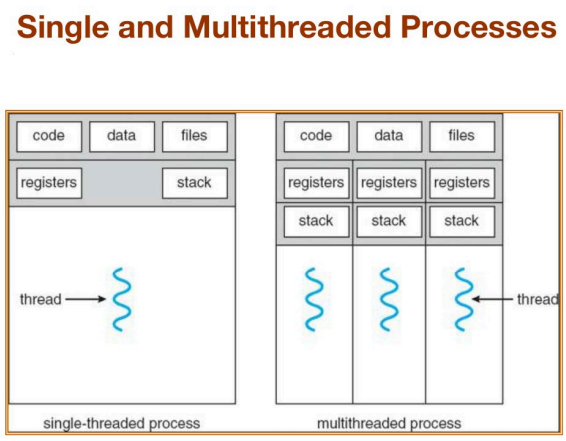
B. Thread & การเขียนโปรแกรม Multi-Threaded (OSNo04)

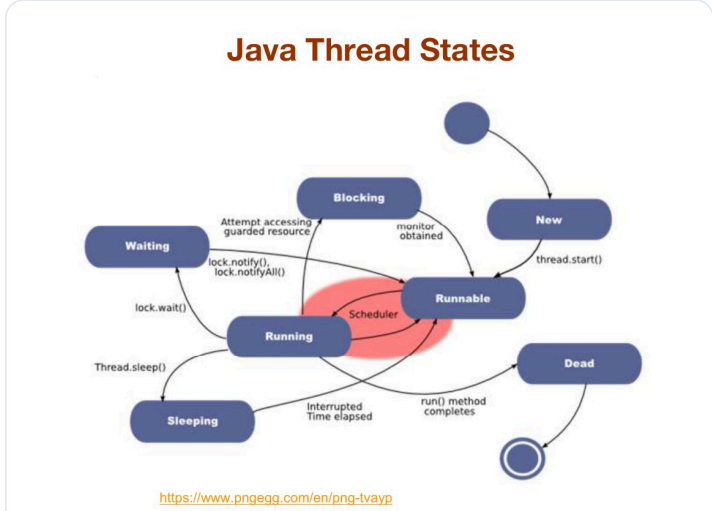
> 📖 **รู้จักส่วนนี้ก่อนอ่าน**
>
> **ความหมาย:** Thread (เธรด/สายงานย่อย) = หน่วยทำงานที่เล็กกว่า process หลาย thread (เธรด (สายงานย่อย)) รันขนานกันภายใน process เดียวกันได้ โดยแชร์หน่วยความจำร่วมกัน
>
> **ความสำคัญกับข้อสอบ:** ถามโมเดลการ map user thread (thread ฝั่งผู้ใช้) → kernel thread (Many-to-One / One-to-One / Many-to-Many) พร้อมตัวอย่าง OS, อ่านโค้ด Pthread/Java ว่าอะไรแชร์กัน และเชื่อมไปบท Synchronization (การประสานงาน)
>
> **ใช้จริงในคอมพิวเตอร์:** เบราว์เซอร์ 1 แท็บใช้หลาย thread (โหลดหน้าเว็บ + เล่นวิดีโอ + รับการคลิก พร้อมกัน) · เกมออนไลน์ (เครือข่าย/กราฟิก/ฟิสิกส์แยกกัน) · Thread Pool (บ่อรวมเธรดเตรียมใช้งาน) ใช้ในเว็บเซิร์ฟเวอร์รับคำขอนับพันพร้อมกัน

B1. Thread คืออะไร

Thread = ลำดับคำสั่งที่รันได้อิสระภายใน process เดียวกัน (lightweight process) · สิ่งที่แต่ละ thread มี**ส่วนตัว**: **program counter, registers, stack** · สิ่งที่**แชร์กัน**: code section, data section, heap, ไฟล์ที่เปิด — thread คือ "หน่วยที่ OS จัดคิวเข้า CPU" ในระบบสมัยใหม่ และ**สลับ thread ถูกกว่าสลับ process**



สไลด์ OSNo04 p.4 — Single vs Multithreaded: ขวาคือแชร์ code/data/files ร่วมกัน แยกแค่ registers+stack ของแต่ละ thread



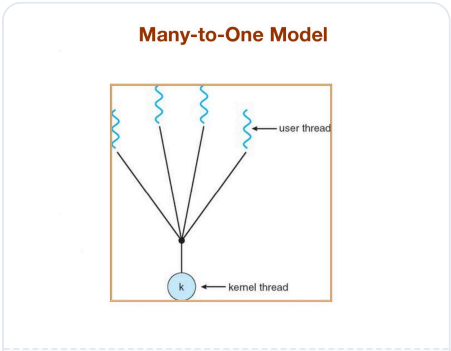
สไลด์ OSNo04 p.25 — Java Thread States (New/Runnable/Running/Blocked/Terminated)

OS แบบไหน	ตัวอย่าง
Single-process single-threaded	**MS-DOS**
Single-process multi-threaded	**JVM**
Multiprocessing single-threaded	Traditional UNIX (ระบบยูนิกซ์)
Multiprocessing multi-threaded	**Modern OS: Windows (ระบบวินโดวส์), Linux (ระบบลินุกซ์), macOS**

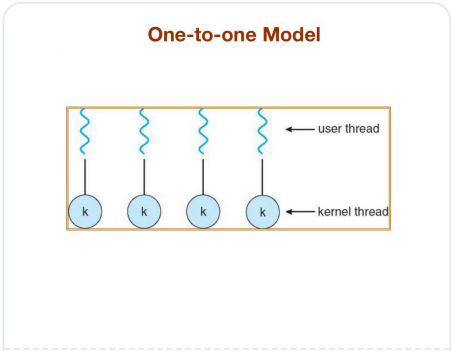
B2. User Thread vs Kernel (แกนหลักของ OS) Thread + Multithreading Models

- **User-Level Thread (ULT)**: จัดการโดย thread library ฝั่งผู้ใช้ kernel ไม่รู้จัก — เร็วมาก แต่ถ้าตัวใด block (บล็อก (หน่วยจัดเก็บ)) ทั้ง process หยุด
- **Kernel-Level Thread (KLT)**: kernel จัดการตรง — block ตัวเดียวตัวอื่นยังรัน ขนานจริงได้

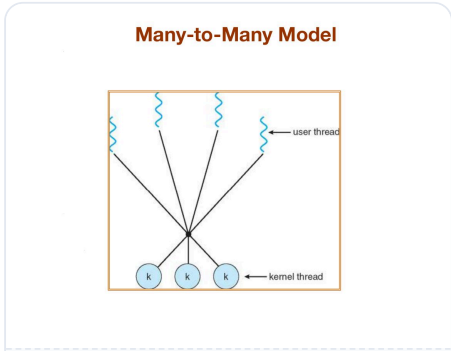
Model	Map	จุดเด่น/จุดพัง	ตัวอย่าง
Many-to-One	ULT หลาย → KLT 1	เร็ว (user space) แต่ block ตัวเดียว = หยุดทั้ง process, ขนานบนหลาย CPU ไม่ได้	Early Unix (ระบบยูนิกซ์)
One-to-One	ULT 1 : 1 KLT	ขนานจริง ตัวใด block ตัวอื่นไม่พัง · สร้าง kernel thread เยอะ = overhead	**Windows NT+, Linux, Solaris 9+**
Many-to-Many	ULT หลาย → KLT หลาย	OS สร้าง KLT ให้พอดีจำนวน	Solaris < 9, Tru64 UNIX



สไลด์ p.14 Many-to-One Model — ULT หลายเส้นรวมเป็น KLT เส้นเดียว



สไลด์ p.16 One-to-One Model — ULT 1 เส้น = KLT 1 เส้น



สไลด์ p.18 Many-to-Many Model — ULT หลายเส้น map ไป KLT หลายเส้น

B3. Pthreads (C) — โค้ดตัวอย่างพร้อมคอมเมนต์

```c
int sum;  // ตัวแปร global — "แชร์" ระหว่าง thread (main กับ runner เห็นตัวเดียวกัน!)

/* ฟังก์ชันที่ thread จะรัน — signature ต้องเป็น void *f(void *) */
void *runner(void *param) {
    int upper = atoi(param);      // รับค่าจาก argv[1] ที่ส่งมาตอน create
    sum = 0;
    for (int i = 1; i <= upper; i++) sum += i;   // หาผลรวม 1..upper
    pthread_exit(NULL);           // จบ thread นี้ (≈ exit ของ process)
}

int main(int argc, char *argv[]) {
    pthread_t tid;                // เก็บ id ของ thread ที่สร้าง
    pthread_attr_t attr;
    pthread_attr_init(&attr);     // ตั้งค่า attribute = default
    pthread_create(&tid, &attr, runner, argv[1]);
    //  ↑ สร้าง thread ให้รันฟังก์ชัน runner โดยส่ง argv[1] เป็น param
    pthread_join(tid, NULL);      // หลักของ main: รอ thread จบก่อน (≈ wait ของ process)
    printf("sum = %d\n", sum);    // เห็นค่า sum ที่ runner แก้ — เพราะแชร์กันจริง
}
/* จับคู่: pthread_create ≈ fork · pthread_join ≈ wait · pthread_exit ≈ exit */
```

Pthreads

```c
int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* the thread */
main(int argc, char *argv[])
{
   pthread_t tid; /* the thread identifier */
   pthread_attr_t attr; /* set of attributes for the thread */
   pthread_attr_init(&attr); /* get the default attributes */
   /* create the thread */
   pthread_create(&tid,&attr,runner,argv[1]);
   /* now wait for the thread to exit */
   pthread_join(tid,NULL);
   printf("sum = %d\n",sum);
}
void *runner(void *param) {
   int upper = atoi(param);
   int i;
   sum = 0;
   if (upper > 0) {
      for (i = 1; i <= upper; i++)
         sum += i;
   }
   pthread_exit(NULL);
}
```

สไลด์ OSNo04 p.19 — Pthreads (โค้ดต้นฉบับ)

Thread Pools

- Instantiate several threads in a pool and have them remain idle, waiting for tasks to be assigned.
- Advantages:
  - Typically, handling a request using an existing thread is more efficient than creating a new one.
  - Enables the application(s) to allocate the number of threads based on the size of the pool.

สไลด์ OSNo04 p.29 — Thread Pools: สร้าง thread เตรียมไว้ใน "บ่อ" รับงานเรื่อย ๆ

Thread Pool (Java 3 แบบ)

Single thread executor (บ่อขนาด 1) · **Fixed thread executor** (ขนาดคงที่) · **Cached thread pool** (ขนาดไม่จำกัด สร้างเพิ่มเอง) — ประโยชน์: ใช้ thread เดิมเร็วกว่าสร้างใหม่ + คุมจำนวน thread ได้

B4. Java Thread — 2 วิธีสร้าง + cancellation

วิธี	โค้ด	หมายเหตุ
extends Thread	`class W extends Thread { public void run() {...} }` → `new W().start()`	เผา slot extends
implements Runnable	`class W implements Runnable { public void run() {...} }` → `new Thread(new W()).start()`	นิยมกว่า

- **Cancellation 2 แบบ: Asynchronous (ยกเลิกทันที)** = ยิงปิดทันที (เสี่ยงข้อมูลครึ่ง ๆ กลาง ๆ) · **Deferred (ยกเลิกแบบให้ตัวมันเช็คเองเป็นระยะ)** = บอกเป้าหมายให้เช็คเองเป็นระยะ (`Thread.currentThread().isInterrupted()`) แล้ว cleanup ก่อนจบ — ปลอดภัยกว่า
- `task.join()` = main รอ task จบก่อนเสมอ (เทียบ wait)

🎯 ประโยคทอง Java scheduling

Java: priority **1–10, default (ค่าเริ่มต้น) 5, มาก = สูง** (Unix ตรงข้าม: น้อย = สูง) · การตั้ง priority **ไม่การันตีลำดับ 100%** — **ห้ามฝากความถูกต้องของ logic ไว้กับ scheduler (ตัวจัดคิว CPU)** ต้องใช้ Synchronization เท่านั้น